SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ITA03 – Mobile Computing: Assignment 4 (CO4)

NAME: Gayathri. P

REG NO:192424324

Task 1 – Architecture Review

Question: A junior engineer submits a MobileOS design note: (a) hardware at the bottom, applications at the top; (b) a kernel in the middle that runs the applications; (c) applications are compiled and executed by the runtime; (d) the OS keeps applications secure using permissions; (e) the OS saves battery when needed; (f) developers use the framework to build applications. Diagnose it, repair it and defend the repair.

Task 1.1 – Diagnose the Description

Question: Identify at least five technical deficiencies in the note and analyse why another engineer could not reliably build or evaluate a system from it.

Answer:

The note is a list of nouns without structure. Eight technical deficiencies are identified below; each one blocks a different kind of implementation or evaluation decision.

#	Statement in the note	Technical deficiency	Why an engineer cannot build or evaluate from it
1	"Hardware at the bottom, applications at the top"	Layers between the two are missing: device drivers, hardware abstraction layer (HAL), native libraries, runtime, application framework with system services.	The engineer cannot tell where a camera driver lives, who owns it, or which layer a vendor must replace when porting to new hardware. Interfaces between layers cannot be defined.
2	"A kernel sits in the middle and runs the applications"	The kernel type (monolithic, hybrid, microkernel) is not stated, and "runs applications" is wrong. The kernel schedules **processes and threads**; it does not run applications. Its real duties (process scheduling, virtual memory, drivers, file systems, networking stack, IPC primitives, security enforcement) are not listed.	Two teams could implement a monolithic Linux-style kernel and a microkernel respectively and both claim to satisfy the note. Fault isolation, driver crash behaviour and performance cannot be predicted.
3	"Applications are compiled and executed by the runtime"	It does not say **what form** of code is delivered (source, bytecode, native binary), **who** compiles it, **when** it becomes machine code (install time, load time, run time), or whether the runtime interprets, JIT-compiles or ahead-of-time compiles.	Start-up time, memory footprint, battery use and portability all depend on this decision. Nothing can be benchmarked or even toolchained from the sentence.
4	"Keeps applications secure using permissions"	Does not say **who grants** permissions (user, system, signature match), **when** (install time or first use), **where enforced** (kernel UID checks,	A tester cannot write a test case: they do not know whether an unauthorised call should fail in the framework,

#	Statement in the note	Technical deficiency	Why an engineer cannot build or evaluate from it
		framework service checks, SELinux policy), nor how apps are isolated (sandbox).	in the kernel or not at all.
5	"Saves battery when needed"	No trigger (screen off, idle timer, low-battery threshold, thermal event) and no mechanism (CPU suspend, frequency scaling, wake locks, background-task scheduling, radio duty-cycling). "When needed" is not measurable.	The behaviour cannot be implemented or verified. There is no state model, no entry or exit condition and no owner for the decision.
6	"Developers use the framework"	The framework is not described: which APIs, which system services (activity, package, notification, location), and how a call travels from the app to the OS.	An application developer does not know the programming model (activities, lifecycle callbacks, intents) or which services may block.
7	(Whole note)	The boundary and communication path between application and OS is absent: system calls, Binder/XPC/COM-style IPC, and the user/kernel privilege boundary are not named.	The security boundary cannot be reasoned about, and nobody knows which calls cross into the kernel and are therefore expensive or protected.
8	(Whole note)	No application lifecycle or process model: how apps start, are paused, killed under memory pressure, or restarted.	Background execution, push delivery and memory-pressure behaviour cannot be designed.

Conclusion of diagnosis: The note names components but gives neither responsibilities, interfaces nor dependencies. It cannot be implemented, tested or compared with another OS.

Task 1.2 – Repair the Description

Question: Choose one OS and rewrite the note as a complete layered architecture specification. For each layer give its name, responsibility, what it exposes upward and what it depends on below. The kernel, runtime, security enforcement point and power management component must each be located explicitly.

Answer (chosen OS: Android):

Android is specified as six layers, bottom to top. Each layer only uses the interface of the layer immediately beneath it, except that the runtime and native libraries use system calls directly.

Layer (bottom → top)	Responsibility	Exposes to the layer above	Depends on (layer below)
L0 Hardware	SoC (CPU cores, GPU, DSP), RAM, flash storage, radios (cellular, Wi-Fi, Bluetooth), sensors, battery and power-management IC.	Registers, interrupts, DMA, memory-mapped I/O.	None (physical).
L1 Linux kernel (monolithic, modified)	Process and thread scheduling (CFS), virtual memory and paging, device drivers (display, camera, Binder, audio), file systems (ext4/f2fs), TCP/IP stack, **Binder IPC driver**, **SELinux and UID-based discretionary access control**, **power management core** (wake locks, CPU idle, cpufreq, suspend), low-memory killer (lmkd).	System-call interface (open, read, mmap, ioctl, fork/clone, socket); /proc and /sys; Binder device node.	L0 through drivers, interrupts and device tree.
L2 Hardware	Standard vendor-implemented interfaces (HIDL/AIDL) that hide device-specific	Stable, versioned HAL interfaces	L1 device nodes and

Layer (bottom → top)	Responsibility	Exposes to the layer above	Depends on (layer below)
Abstraction Layer (HAL)	drivers: camera, audio, sensors, radio, graphics, power HAL.	called by system services.	ioctl calls.
L3 Native libraries and Android Runtime (ART)	Native libraries: bionic libc, SQLite, OpenSSL/BoringSSL, WebKit/Chromium WebView, Skia, OpenGL ES/Vulkan, media codecs. **ART runtime** executes app bytecode (DEX): ahead-of-time compilation at install, JIT and profile-guided compilation, garbage collection, and JNI bridging to native code.	JNI interface to native code; the Java/Kotlin core libraries; native APIs through the NDK.	L1 system calls (through bionic) and L2 HALs.
L4 Android Framework (system services)	Java/Kotlin APIs and system services in system_server: ActivityManager (lifecycle, process priorities), PackageManager (install, signatures, **permission grants**), WindowManager, NotificationManager, PowerManager and JobScheduler/WorkManager (**power policy**), LocationManager, ConnectivityManager.	Public SDK API: Activity, Service, BroadcastReceiver, ContentProvider, Intents.	L3 runtime and L2 HAL; talks to apps and between services over Binder (L1).
L5 Applications	User apps and system apps, each in its own process with a unique Linux UID (application sandbox).	User interface and features.	L4 SDK APIs only.

Explicit location of the four required components:

Required component	Where it is located	How it works
Kernel	L1, the Linux kernel (monolithic with loadable modules).	Provides scheduling, memory, drivers, IPC and security primitives to everything above.
Runtime	L3, Android Runtime (ART), running inside each app process (forked from the Zygote process).	Loads DEX code from the APK, AOT/JIT compiles it to ARM machine code, manages the heap with garbage collection.
Security enforcement point	Two cooperating points. (1) Kernel L1: per-app UID sandbox, file permissions, SELinux mandatory access control, seccomp filters. (2) Framework L4: permission checks done by system services (PackageManager and the service being called) via the caller's UID obtained through Binder.	Permission is granted by PackageManager/user at install or first use; enforcement happens on every call at the service or the kernel.
Power management component	L4 PowerManagerService (policy: screen, doze, app standby, battery saver, JobScheduler) and L1 kernel power core (mechanism: wake locks, cpuidle, cpufreq, suspend-to-RAM), with the power HAL in L2 between them.	Policy decides *when* to enter low-power states; the kernel executes the state change.

Boundary and communication path: An app calls an SDK API (L4). The framework proxy sends a **Binder transaction** through the kernel to system_server. System_server checks the caller's UID and permissions, then calls a HAL (L2), which calls the driver through **ioctl** into the kernel (L1) and finally reaches the hardware (L0). The user/kernel privilege boundary is crossed only by system calls and Binder.

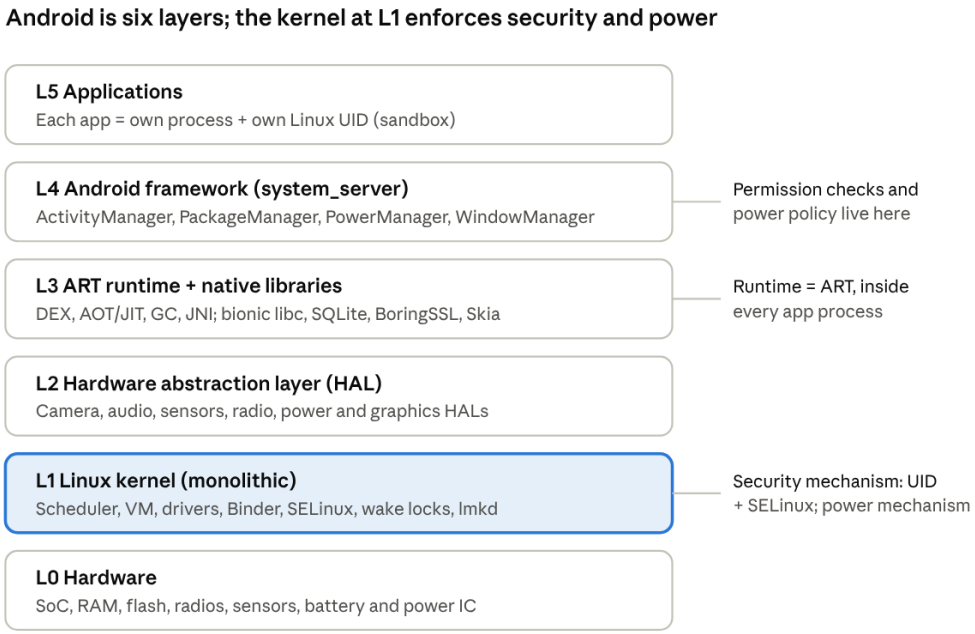


Figure 1 · Android layered architecture · 6 layers

Figure 1. The layered specification of Task 1.2. The kernel (highlighted) is the single place where process isolation, SELinux and wake locks are enforced; the runtime and the framework policy sit above it.

Task 1.3 – Design Defence

Question: In not more than 100 words, analyse why the rewritten specification is a precise architectural description rather than a marketing overview or informal English.

Answer (96 words):

The rewrite is precise because every layer has four testable attributes: responsibility, upward interface, downward dependency and owner. The kernel type, runtime (ART), enforcement points (SELinux/UID in the kernel, permission checks in framework services) and power components (PowerManagerService policy, kernel wake-lock mechanism) are located exactly. Interfaces are named concretely: system calls, Binder, HAL, JNI. An engineer could write a test, such as "an unprivileged UID calling a protected service must be denied", and know which layer fails. Marketing statements such as "secure" or "efficient" cannot be tested or implemented.

Evidence provided: Eight-point diagnosis, six-layer specification with the four components located, and a 96-word defence.

Task 2 – From Platform Requirement to Architectural Comparison

Scenario: A start-up builds a secure messaging application that must receive messages in the background on Android, iOS and Windows. The team must understand how kernel structure and runtime will affect the product.

Task 2.1 – Model the Requirement

Question: Identify input, output, objective, constraints, decision and the architectural information needed.

Answer:

Element	Statement
Input	Three platform architectures: Android (Linux kernel, ART runtime), iOS (Darwin/XNU kernel, Objective-C/Swift runtime), Windows (NT kernel, WinRT/Windows Runtime).
Output	A comparison matrix with a consequence for the messaging app for each architectural difference, and a ranked list of platform risks.
Objective	Deliver each incoming message to the user quickly, securely and without draining the battery, on all three platforms.
Constraints	(1) Background execution: the OS may suspend or kill the app. (2) Security: message content must stay confidential, so isolation and encrypted storage are needed. (3) Battery: no continuous wake-ups or

Element	Statement
	polling. (4) Responsiveness: delivery and display within about one second of arrival when the user opens the device.
Decision to be made	Which background-delivery design to use on each platform (OS push service with a short wake-up, versus persistent connection), and how much code can be shared across the three platforms.
Architectural information needed	Kernel family and structure; kernel programming interface; runtime and when code is compiled; application framework and lifecycle; isolation model; IPC mechanism; background-execution rules; push service; power management behaviour.

Task 2.2 – Formulate a Hypothesis

Answer: I expect the **process model and background-execution policy of the framework and runtime (app suspension and termination rules) to affect background message delivery the most**, because kernel structure changes performance marginally, whereas the OS suspending or killing the app process decides whether any app code can run when a message arrives.

Task 2.3 – Build the Comparison

Question: Prepare a comparison matrix. Every cell must be a specific technical statement; add the consequence for the messaging app.

Answer:

Aspect	Android	iOS	Windows
Kernel family and structure	Linux kernel, monolithic with loadable modules;	Darwin/XNU: hybrid of Mach microkernel primitives (tasks,	Windows NT kernel (Windows 8 era): hybrid kernel with

Aspect	Android	iOS	Windows
	drivers run in kernel mode.	threads, ports, VM) and BSD subsystem (POSIX, networking, VFS) in a single kernel address space.	executive services, kernel-mode drivers, and a Hardware Abstraction Layer (HAL).
Kernel-level programming interface	Linux syscalls through bionic libc; native code via NDK and JNI; ioctl on driver nodes.	BSD syscalls and Mach traps through libSystem; native code in C/C++/Objective-C directly; Mach messages.	Win32/NT system calls through ntdll.dll; native C++ with Win32 and WinRT components.
Runtime environment	Dalvik (older) or ART: DEX bytecode, GC heap, JNI bridge.	Objective-C runtime on native machine code (Swift runtime in newer versions); reference counting (ARC); no VM.	WinRT (Windows Runtime): COM-based object model with language projections for C++/CX, C# (CLR) and JavaScript.
Application framework	Android SDK: Activity, Service, BroadcastReceiver, ContentProvider, Intent.	Cocoa Touch: UIKit, Foundation, UIApplicationDelegate lifecycle, GCD.	WinRT APIs: XAML UI, Application class, background tasks with triggers.
Application isolation model	Each app has its own Linux UID and process; SELinux domains; per-app private data directory.	Per-app sandbox container enforced by the kernel Sandbox (Seatbelt/MACF) with entitlements; code	AppContainer sandbox with a low-integrity token; capability declarations in the app manifest.

Aspect	Android	iOS	Windows
		signing mandatory.	
Inter-process communication	Binder (kernel driver) with UID identification in each transaction; also Intents and sockets.	Mach ports and XPC services; app extensions.	COM/RPC and ALPC (Advanced Local Procedure Call); contracts and Share/Search.
Background execution	Services, JobScheduler/WorkManager, Doze; app may be killed by lmkd. High-priority FCM message can start the app briefly.	No arbitrary background running; APNs push, silent notification, a limited background fetch window; app is suspended after leaving the foreground.	Background tasks only through triggers (push, timer, system events), with CPU and network quotas; the app is suspended (Windows 8).

Consequence for the messaging application:

Difference	Consequence for the messaging app
Linux monolithic kernel vs Darwin hybrid vs NT hybrid	Little effect on app code; the kernel difference mainly changes native-code portability and driver behaviour. A shared native crypto core in C/C++ can be compiled for all three.
Runtime: ART (bytecode, GC) vs Objective-C (native, reference counting) vs WinRT (COM)	The message-handling code cannot be shared directly in one language; it needs a platform adaptation layer around a shared C/C++ core. GC pauses on Android can delay message rendering.
Isolation model	All three prevent other apps from reading the local message database, but the keys must be

Difference	Consequence for the messaging app
	stored in the platform keystore (Android Keystore, iOS Keychain, Windows Credential Locker/DPAPI).
IPC differences	Share and notification extensions use different mechanisms, so the notification handling code must be written per platform.
Background execution (largest effect)	The app cannot hold its own connection on iOS and Windows; it must rely on **push notifications** (FCM, APNs, WNS), with the message payload encrypted end-to-end so the push server only forwards ciphertext.

Task 2.4 – Desk-Check

Question: Trace one incoming message with the screen off through two platforms (Android and iOS chosen).

Answer:

Step	Layer	Android (FCM)	iOS (APNs)
1	Network event	Wi-Fi/cellular radio receives a packet on the connection kept by Google Play Services.	Radio receives a packet on the connection kept by the apsd (Apple Push) system daemon.
2	Kernel/driver	Modem/Wi-Fi driver raises an interrupt and wakes the CPU from idle; the TCP stack delivers data to the socket.	Same: interrupt, driver, then the BSD networking stack in XNU delivers the data.
3	System service	Google Play Services (FCM client) receives the message and	apsd receives the payload and hands it to SpringBoard or the

Step	Layer	Android (FCM)	iOS (APNs)
		asks ActivityManager to deliver it, using a high-priority wake allowance.	target app through a Mach/XPC call.
4	Runtime	If the app is not running, Zygote forks a new process, ART loads DEX code and the app's FirebaseMessagingService is created.	If the payload has content-available, the OS may launch the app in the background; the Objective-C runtime loads the binary. Otherwise the system displays the notification itself.
5	App callback	onMessageReceived() decrypts the message and writes it to the local database.	application:didReceiveRemoteNotification:fetchCompletionHandler: (or a Notification Service Extension) decrypts and stores the message.
6	System UI	NotificationManager shows the notification; the CPU returns to idle after the wake window ends.	SpringBoard shows the banner; the app returns to suspended state.

Task 2.5 – Challenge Your Comparison

Changed condition: The app process has been **terminated by the OS due to memory pressure**.

1. **Prediction:** Delivery will still work on both platforms, but through different paths. Android will cold-start the process to handle a high-priority FCM message; iOS will deliver only a visible notification unless the app is allowed to launch for a silent push, and it will not launch an app the user force-quit.

2. **Architectural element responsible:** The process-lifecycle and push-delivery services of the framework (ActivityManager with FCM on Android; apsd with SpringBoard on iOS), together with the kernel's low-memory killer, which removed the process.
3. **Relevant steps traced:** Android: step 3 (FCM client in Google Play Services is a separate, still-running process) then step 4 (Zygote fork, ART start-up, a cold start), then step 5. iOS: step 3 (apsd is a system daemon, still alive) then a visible alert from the system; the app code runs only when the user taps it or when the system grants a background launch.

Result: The prediction was **correct**. The difference lies in the process model, which supports the hypothesis of Task 2.2.

Evidence provided: Problem model, hypothesis, specific matrix with consequences, layer-wise trace and modified-case reasoning.

Task 3 – Runtime Environment Analysis

Scenario: A team wants to understand exactly what happens between a user tapping an application icon and the first screen appearing on Android (Dalvik/ART), iOS (Objective-C runtime) and Windows (WinRT).

Task 3.1 – Define the Execution Model (1 Mark)

Question: For each platform define what form of code is delivered, who loads it, and when it becomes executable machine instructions.

Answer:

Platform	Form of code delivered to the device	Who loads it	When it becomes machine instructions
Android (Dalvik)	APK package containing **DEX bytecode** (Dalvik Executable, register-	The Zygote process forks a new process; the Dalvik VM (or ART) in that process loads the	**Run time** in Dalvik (interpreter plus trace-based JIT, introduced in Android 2.2). In ART (Android 5+),

Platform	Form of code delivered to the device	Who loads it	When it becomes machine instructions
	based), resources and a signed manifest.	DEX through a class loader.	mostly **install time** through dex2oat ahead-of-time compilation, with JIT and profile-guided compilation later.
iOS (Objective-C runtime)	IPA package containing a signed **native Mach-O binary** compiled ahead of time for ARM, plus resources and entitlements.	The kernel's loader (XNU, executing dyld, the dynamic linker) maps the binary and its shared libraries; the Objective-C runtime then registers classes and selectors.	**Build time** on the developer's machine (native ARM code). The device does not compile; machine instructions exist before installation. Pages are mapped at **load time**.
Windows (WinRT)	An APPX package. Native C++ apps ship as **native x86/ARM binaries**; C# apps ship as **.NET IL (MSIL) assemblies**; JavaScript apps ship as source. All call WinRT APIs described by metadata (.winmd).	The Windows loader (ntdll) maps native code; for managed code the CLR loads assemblies; for JavaScript the Chakra engine loads script. The WinRT activation factory creates runtime classes.	Native: at **build time**. C# IL: **install time/load time** through NGEN pre-compilation or **run time** through JIT. JavaScript: run time by interpretation and JIT.

Task 3.2 – Develop the Launch Sequence (2 Marks)

Question: Develop the ordered launch sequence for one platform, from user action to first rendered screen, stating what each step does, which component performs it, what must already exist, and its purpose.

Answer (platform chosen: Android with ART):

Step	What happens	Component	Must already exist	Purpose of the step
1	User taps the icon; a touch event is read from the touchscreen and sent to the launcher app.	Touch driver (kernel), InputDispatcher in system_server, Launcher app	Kernel input drivers, running system_server	Translates a physical action into an intent to start an app.
2	The launcher builds an **Intent** (ACTION_MAIN, CATEGORY_LAUNCHER) and calls startActivity, which goes through Binder to ActivityManagerService.	Launcher, Binder driver, ActivityManager Service (AMS)	Binder IPC, installed manifest information held by PackageManager	Names the target component and crosses the process boundary safely.
3	AMS resolves the Intent through PackageManager and checks permissions and that the app is installed and enabled.	AMS, PackageManager	Package database, signature records	Ensures the caller is allowed and the target exists.
4	AMS checks whether a process for the app exists. If not, it asks the **Zygote** process to fork a new process using a UNIX socket.	AMS, Zygote, kernel fork()	Pre-warmed Zygote with the framework classes preloaded	Fast start: a copy-on-write fork reuses preloaded classes and resources instead of initialising the

Step	What happens	Component	Must already exist	Purpose of the step
				runtime from nothing.
5	The kernel creates the process with the app's **UID**, applies the SELinux domain and seccomp filter and sets its memory limits.	Linux kernel	Zygote; app UID assigned at install	Establishes the security sandbox before any app code runs.
6	The new process starts **ActivityThread.main()**, creates the Looper (main thread message loop) and attaches to AMS through Binder.	ART runtime, framework (ActivityThread)	Runtime initialised (inherited from Zygote), Binder thread pool	Gives the app a main thread and a link to the system.
7	AMS sends bindApplication; the runtime opens the APK, loads the compiled code (OAT or DEX with JIT) and creates the **Application** object (onCreate).	ART class loader, ActivityThread	APK on storage, oat/vdex files from install-time compilation	Loads the app's own code, and lets it initialise global state.
8	AMS schedules the launch Activity; the framework instantiates it and calls **onCreate(), onStart(), onResume()**.	ActivityThread, Instrumentation, AMS	Application object, resources, theme	Runs the app's code in the lifecycle order the framework guarantees.
9	The Activity inflates its layout and registers a window with	View framework,	Display HAL, graphics buffers	Creates the surface on

Step	What happens	Component	Must already exist	Purpose of the step
	WindowManager; a surface is allocated through SurfaceFlinger.	WindowManagerService, SurfaceFlinger, graphics HAL and driver		which the screen can be drawn.
10	The first frame is measured, laid out, drawn (Skia/HWUI, GPU) and **composited**; the display shows it.	ViewRootImpl, RenderThread, SurfaceFlinger, display driver	GPU driver, vsync signal from the display	Produces the first visible screen.

Task 3.3 – Compare Runtime Strategies (4 Marks)

Question: Analyse interpretation, JIT and AOT; memory management; reach to native code and kernel; and effects on start-up, footprint and portability; conclude which is best for a low-memory device.

Answer:

(a) Interpretation, JIT and AOT, and which platform uses which

Strategy	Principle	Used by	Effect
Interpretation	Each bytecode instruction is decoded and executed by software one at a time; no machine code is stored.	Early Dalvik and the first run of cold code in ART; WML/J2ME on very small devices (KVM interpreter).	Smallest memory use and fastest first start, but slowest steady execution and highest CPU energy per operation.
JIT	Hot code is compiled to	Dalvik (from Android 2.2); ART JIT since Android 7;	Good steady speed after warm-up, but compile time

Strategy	Principle	Used by	Effect
	machine code at run time and cached in memory.	CLR JIT for C# on Windows; JavaScript engines.	and the code cache use CPU and RAM and cost battery at launch.
AOT	Source or bytecode is compiled to native code before running.	iOS (native Mach-O compiled on the developer machine); Windows native C++; ART dex2oat at install (Android 5 to 6) and profile-guided AOT (Android 7+).	Fastest start and no run-time compiler, but larger storage footprint and slower install; the code is tied to one CPU architecture.

(b) Memory management

- **Dalvik/ART use tracing garbage collection** (mark-sweep in Dalvik; concurrent copying collector in ART). The collector finds unreachable objects automatically, but it needs headroom (heap about twice the live data to run efficiently) and can pause the application. Dalvik's stop-the-world pauses caused visible jank; ART's concurrent collector shortened them. A GC cycle consumes CPU, so it affects both responsiveness and battery.
- **iOS uses reference counting (ARC, Automatic Reference Counting).** The compiler inserts retain/release calls; memory is freed immediately when the count reaches zero, so there are no collector pauses and the footprint is tight. Cost: small constant overhead per assignment and the programmer must avoid retain cycles using weak references.
- **WinRT** uses reference counting for its COM-based objects (AddRef/Release); managed C# code in the same app uses the CLR garbage collector, so a C# app has both.

(c) Reaching native code and kernel services

Platform	Path from the runtime to native code and kernel
Android	**JNI** (Java Native Interface) lets bytecode call C/C++ libraries (NDK);

Platform	Path from the runtime to native code and kernel
	those libraries call the kernel through bionic libc system calls.
iOS	Objective-C code is already native: messages go through objc_msgSend into C functions and directly into libSystem, which issues BSD syscalls or Mach traps. No bridge is needed.
Windows	C++ calls Win32 and WinRT directly; C# uses **P/Invoke** and the WinRT projection; ntdll.dll issues the NT system calls.

(d) Effect on start-up, memory footprint and portability

Property	Android (Dalvik/ART)	iOS (Objective-C, native)	Windows (WinRT)
Start-up time	Fast process creation by Zygote fork; Dalvik slower to warm up (interpreter and JIT); ART AOT improves it.	Fastest: native code is mapped directly; the cost is dyld linking.	Native is fast; C# adds CLR start-up and JIT unless pre-compiled.
Memory footprint	Highest: a VM per process and GC headroom; Zygote sharing reduces it through copy-on-write.	Lowest: no VM, deterministic freeing by ARC.	Medium: native low, managed higher because of CLR plus GC.
Portability	Highest: the same DEX runs on ARM, x86 and MIPS devices, and across vendors.	Lowest: binaries are compiled for the ARM architecture and Apple's frameworks.	Medium: IL and JavaScript are portable between Windows architectures; native C++ is not.

Conclusion – best design for a low-memory device: Ahead-of-time compiled native code with reference counting (the iOS/Objective-C style) is most suitable, because it needs no

virtual machine, no JIT cache and no garbage-collector headroom, and objects are freed at the moment they are no longer used. Where portability across hardware is also required, an ART-style design with install-time AOT compilation and a concurrent collector is the best compromise; a pure interpreter is acceptable only on extremely small devices such as J2ME feature phones.

Task 3.4 – Desk-Check: Cold Start and Warm Start

Question: Using the launch sequence of Task 3.2, trace a cold start and a warm start and show which steps run and which are skipped.

Answer:

Step (from Task 3.2)	Cold start (no process, no cached state)	Warm start (process alive, Activity was destroyed)	Hot start (process and Activity alive in background)
1 Touch event handled	Runs	Runs	Runs
2 Intent sent to AMS	Runs	Runs	Runs
3 Resolve and permission check	Runs	Runs	Runs
4 Zygote fork	**Runs** (process created)	**Skipped**: the process exists	Skipped
5 Kernel sandbox set-up	**Runs**	Skipped: already applied	Skipped
6 ActivityThread.main(), Looper	**Runs**	Skipped	Skipped
7 Load code, Application.onCreate	**Runs** (code may be loaded from storage)	Skipped: the Application object persists	Skipped

Step (from Task 3.2)	Cold start (no process, no cached state)	Warm start (process alive, Activity was destroyed)	Hot start (process and Activity alive in background)
8 Activity lifecycle	onCreate, onStart, onResume	onCreate, onStart, onResume (new Activity instance, saved state restored)	Only onRestart, onStart, onResume
9 Window and surface	Runs	Runs (new window)	Often the existing surface is reused
10 First frame	Slowest (about the longest path, with disk reads and class loading)	Faster	Fastest

Steps 4 to 7 are skipped in warm and hot starts because they only create what already exists in memory (the process, runtime state and Application object).

Task 3.5 – Analyse the Dependency

Question: Analyse why certain components must be available before later steps, relating the explanation to the launch order.

Answer:

The launch sequence is a **dependency chain**: each step needs an output of the earlier step.

1. **Kernel process and memory services (step 5)** must exist first because an application's code needs an address space, a scheduled thread and a UID before it can execute. Without the process, there is nowhere to load code and no identity on which to enforce permissions.

2. **Runtime initialisation (steps 4 and 6)** must precede application code because the bytecode cannot run without a class loader, heap, garbage collector and JNI table. Zygote supplies these in advance, which is why launching is fast.
3. **Binder and system services (steps 2 and 3)** must exist before the process is created because the app attaches to AMS through Binder to learn which component to start.
4. **Framework callbacks (step 8)** depend on steps 6 and 7: onCreate() is invoked on the main thread's Looper by ActivityThread and needs the Application object and the resources.
5. **Drawing (steps 9 and 10)** depends on the Activity existing and on WindowManager, SurfaceFlinger and the GPU driver being ready.

If the order is violated, for example calling onCreate() before the runtime is initialised, there is no heap or class to instantiate and the launch fails.

Evidence provided: Execution model definition, ten-step justified launch sequence, runtime comparison, cold/warm/hot trace and dependency explanation.

Task 4 – Security Forensics

Scenario: SecureNotes has four security decisions: (1) at install it requests every available permission "for future use"; (2) the login token is saved in a file readable by other applications; (3) updates are accepted from any source without checking who produced them; (4) all server communication uses plain, unencrypted connections. Chosen platform for repair: **Android**.

Task 4.1 – Locate the Defects (2 Marks)

Question: For each decision identify the OS security mechanism that is bypassed or unused, and analyse why the OS cannot protect the user.

Answer:

#	Design decision	Mechanism bypassed or unused	Why the OS cannot protect the user

#	Design decision	Mechanism bypassed or unused	Why the OS cannot protect the user
1	Requests every permission at install "for future use".	**Permission model** (least privilege; user consent per capability).	The OS grants exactly what the manifest declares. Once granted, every call by SecureNotes to the contacts, SMS, camera, location or storage service passes the permission check, so the OS treats it as legitimate. A vulnerability or malicious library inside the app now holds all those rights, and the user is unable to judge a request that asks for everything.
2	Login token stored in a file readable by other applications.	**Application sandbox** (separate UID and private data directory, SELinux) and **secure storage** (Android Keystore, EncryptedSharedPreferences).	The sandbox protects only files that the app keeps private. A world-readable file (MODE_WORLD_READABLE, or in shared external storage) is explicitly marked as readable by every UID, so the kernel's discretionary access check allows it. The OS honours the app's own permission setting.
3	Accepts updates from any source without checking the producer.	**Code signing** (APK signature scheme v2/v3 and the same-signature requirement for updates).	PackageManager rejects an update that is signed with a different key than the installed app, but only when the update is installed through the normal path. If SecureNotes downloads and installs code from an unchecked source (or loads dynamic code), no signature is verified, so the OS cannot know the code is not from the developer.
4	Plain, unencrypted	**Transport protection** (TLS,	The OS provides TLS libraries (BoringSSL) and the network security configuration to

#	Design decision	Mechanism bypassed or unused	Why the OS cannot protect the user
	connections for all server traffic.	network security configuration, certificate validation).	block cleartext. The app chooses to use http:// so the data crosses Wi-Fi and the Internet as readable text. The OS cannot encrypt a connection the app has not asked to be secure.

Task 4.2 – Construct an Attack Scenario (2 Marks)

Question: Design a realistic scenario in which a malicious application or network attacker exploits at least two defects, trace it step by step, and show where confidentiality or integrity is first lost.

Answer – "Coffee-shop attack" combining defects 4 (cleartext) and 2 (readable token), plus defect 3 (unsigned update):

Step	Actor	Action	State of security
1	User	Connects to the public Wi-Fi "CafeFreeWiFi" and opens SecureNotes, which logs in with its saved token.	–
2	Attacker	Runs a rogue access point and a packet sniffer on the same network (a network attacker).	–
3	App	Sends `GET http://notes.example.com/sync` with the header `Authorization: Bearer <token>` in cleartext.	**Confidentiality is first lost here:** the bearer token travels as readable text and the attacker captures it.
4	Attacker	Replays the token from another device to	Confidentiality of all notes is lost; no

Step	Actor	Action	State of security
		read the user's notes through the API.	password was needed.
5	Attacker	Answers the app's "check for update" request (also over http) with a modified APK or code file.	**Integrity lost:** the app accepts it because no signer check exists.
6	Malicious update	Runs with all permissions granted at install (defect 1), reads contacts and SMS, and uploads them.	Wider damage because the permissions were over-granted.

Second path (local malicious app): a malicious flashlight app installed on the same phone lists the files in SecureNotes's directory. Because the token file is readable by other applications (defect 2), the flashlight app reads it and calls the server. Confidentiality is first lost at the instant the malicious app opens the world-readable file; no network is involved.

Task 4.3 – Repair the Design (2 Marks)

Question: Correct only the required decisions. Name the OS mechanism relied on and where it appears in the OS architecture.

Answer:

#	Minimal repair	OS mechanism relied on	Where it appears in the Android architecture
1	Declare only the permissions needed now (for example READ_CONTACTS for contact search) and request them at run time, when the feature is first	**Runtime permission model**	Granted and recorded by **PackageManager/PermissionController (L4 framework)**; enforced when the app calls a system service: the service checks the caller's UID,

#	Minimal repair	OS mechanism relied on	Where it appears in the Android architecture
	used; the user can deny or revoke them.		delivered by Binder from the **kernel (L1)**.
2	Store the token in **EncryptedSharedPreferences** backed by the **Android Keystore**, in the app's private directory (MODE_PRIVATE), not on shared storage.	**Application sandbox and hardware-backed key storage**	Sandbox: **Linux UID and SELinux in the kernel (L1)**. Keystore: keystore2 system service (L4) using the Trusted Execution Environment/StrongBox through the **Keymaster/KeyMint HAL (L2)**.
3	Remove self-installed updates; distribute updates only through the app store (Google Play) and rely on same-key signature checking by the OS.	**Code signing**	**PackageManagerService (L4)** verifies the APK signature (v2/v3) and refuses an update signed by a different key than the installed one.
4	Use HTTPS (TLS 1.2 or higher) for every call; set android:usesCleartextTraffic="false", using a network security configuration; optionally pin the server certificate.	**Transport protection**	TLS in **BoringSSL (native library, L3)**; policy enforced by the network stack in the **kernel (L1)** and the framework's network security configuration (L4).

Only the four decisions changed; SecureNotes's notes features, user interface and data model remain as they were.

Task 4.4 – Regression Check (1 Mark)

Question: Identify one legitimate behaviour that must still work and show that the repairs preserve it.

Answer: Behaviour: the user searches contacts to share a note and the app syncs it to the server.

Check	Before the repair	After the repair	Preserved?
Read contacts after consent	The permission was granted at install.	At first use of "share with contact", the app calls requestPermissions(READ_CONTACTS); the user taps **Allow**; PermissionController records the grant.	Yes: after consent, the same ContentResolver query returns contacts.
Read the login token on next launch	Plain file read.	Reads from EncryptedSharedPreferences; the Keystore key decrypts it inside the app's own process.	Yes: the same app, same UID, same key.
Sync a note	HTTP POST to the server.	HTTPS POST to the same endpoint; the server already supports TLS (or is configured to).	Yes: same payload and the same response.
Receive an update	Any source.	Installed through the store; the signature matches.	Yes: the user still gets updates.

If the user denies the contacts permission, only that feature is disabled; the rest of SecureNotes continues to work. This is the intended behaviour of the repaired design.

Evidence provided: Four defects with named mechanisms, a student-designed attack trace with the first point of loss, four minimal repairs and a regression table.

Task 5 – Power Management Reasoning

Scenario: A fitness application keeps the processor awake permanently so that it never misses a sensor reading. The battery drains within a few hours even when the phone is untouched. (Platform reasoning: Android, with equivalent mechanisms on iOS and Windows.)

Diagnosis of the fault: The app holds a **partial wake lock** permanently. The kernel's power core counts the lock as a reason not to suspend, so the CPU never enters its deep idle or suspend-to-RAM state, and sensors, radios and the scheduler stay active even when the screen is off.

Task 5.1 – Design the Power-State Policy

Question: Design a power-state model with at least three states. For each: what hardware and kernel do, the entry condition, the exit condition, and which applications or services may run. Present a state-transition table.

Answer: Four device states are defined.

State	Hardware and kernel behaviour	Entry condition	Exit condition	What may run
S0 Active	Screen on; all CPU cores available; cpufreq governor raises frequency on demand; radios on; GPU active.	User interaction (touch, power button), incoming call or alarm.	No input for the screen timeout (for example 30 s) → S1.	Foreground app, all services, all allowed background work.
S1 Idle (screen off, awake)	Screen off; CPU in cpuidle shallow states; frequency lowered; sensors may be batched by a sensor hub; radios keep connection.	Screen timeout or the power button pressed.	No wake lock held and no activity for a short interval (about a minute) → S2; user wakes the device → S0.	Services holding a wake lock; the music player; the sensor hub for step counting; scheduled jobs in their window.

State	Hardware and kernel behaviour	Entry condition	Exit condition	What may run
S2 Doze (deep idle)	CPU in deep idle; network access deferred; alarms and jobs deferred to periodic maintenance windows; sensors batched in hardware.	Device stationary, screen off and unplugged for a period (for example 30 minutes or longer).	Motion detected, screen on, charger connected, or a maintenance window opens → S1 or S0.	Only high-priority push messages and system services; apps are restricted to short maintenance windows.
S3 Suspended (suspend-to-RAM)	Most of the SoC powered down; RAM kept in self-refresh; only the modem, wake-up interrupt controller and the real-time clock are powered.	No wake locks held, no pending work, and the last activity finished (the kernel's autosleep).	A wake interrupt: power button, incoming call or push packet from the modem, alarm from the RTC, or a charger event → S1 or S0.	Nothing (the CPU is stopped). Only the hardware wake sources can cause an exit.

State-transition table:

From	Event	To	Action
S0	Screen timeout / power button	S1	Turn display off; release screen wake lock.
S1	No wake locks held and idle timer fires	S2 or S3	Defer jobs; batch sensors; kernel autosleep.
S1	Touch or power button	S0	Turn display on; raise CPU frequency.

From	Event	To	Action
S2	Maintenance window opens	S1 (temporary)	Run deferred jobs and syncs; then return to S2.
S2 / S3	High-priority push, alarm, call	S1	Wake the CPU for a short, bounded period.
S2 / S3	User wakes the device	S0	Restore full operation.
Any	Charger connected	S0 or S1	Leave Doze and relax restrictions.

The fitness app must therefore stop holding a permanent wake lock: step and heart-rate data come from the **sensor hub with batching**, and the app reads the batch when it is woken.

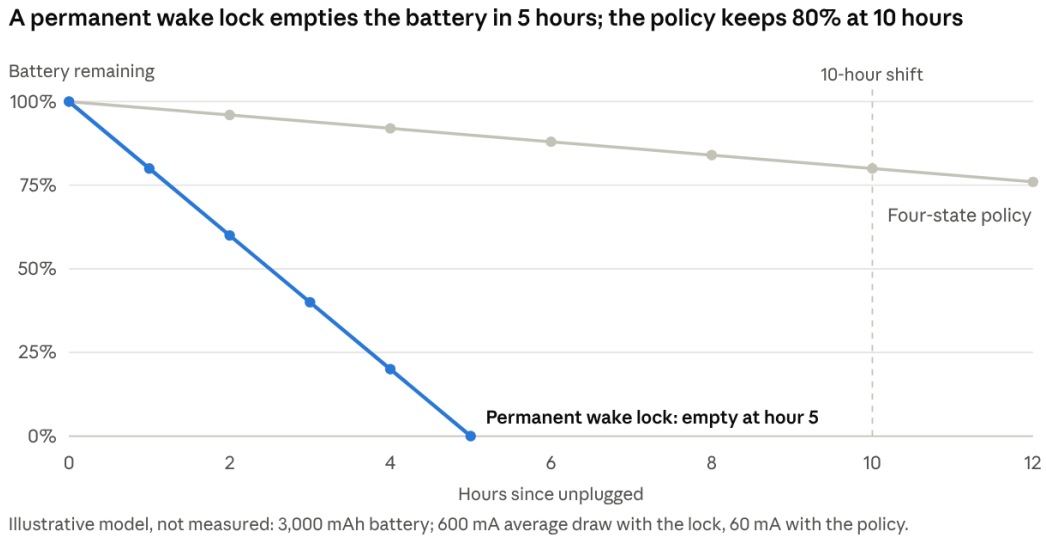


Figure 2 · illustrative battery model (assumed currents) · 0–12 h

Figure 2. Graph of the fault and its repair. The line for the permanent wake lock falls steeply because the CPU never suspends; with the four-state policy most of the time is spent in S2 and S3, so the shift ends with most of the charge left. The currents are assumptions chosen to show the shape, not measurements.

Task 5.2 – Change the Assumption

Question: The app must also receive urgent push notifications while the device is suspended. Identify what becomes unsafe or insufficient; analyse the mechanism that lets the device stay mostly asleep; say what the app must stop doing.

Answer:

What becomes unsafe or insufficient: The rule "in S3 nothing runs" is insufficient, because the app now needs a wake-up path. The previous design assumption that only hardware events wake the device does not say who owns the connection that detects a push. The unsafe alternative, keeping the CPU awake with a permanent wake lock, or the app polling the server, would defeat the policy.

How the platform mechanism solves it:

1. The device keeps **one shared connection** to the platform push service (Firebase Cloud Messaging on Android, APNs on iOS, WNS on Windows), maintained by a system process and the **modem**, not by each app.
2. While suspended, the modem's low-power core watches the connection. A **push packet raises a wake interrupt** and the kernel resumes the CPU (S3 → S1).
3. The system service delivers the message to the app, and the system grants a **short wake allowance** (about a few seconds) so the app can fetch or decrypt the payload.
4. The app finishes and releases its resources; the kernel autosleep returns the device to S3.

Bulk, non-urgent work uses **scheduled background tasks** (JobScheduler/WorkManager) that the OS batches into maintenance windows, so the radio and CPU wake together once instead of many times.

What the application must stop doing: (a) holding a permanent wake lock; (b) polling on its own timers with AlarmManager.setExact for non-urgent work; (c) keeping its own TCP connection open in the background; (d) running a service that never stops. Instead it registers for push, uses batched sensor reads, and holds a wake lock only briefly with an explicit release and timeout.

Task 5.3 – Change the Requirement

Question: A battery-saver mode must activate at 15% charge. Define what changes at the kernel, runtime and application level, and state one user-visible trade-off with justification.

Answer:

Level	Changes when battery saver activates at 15%
Kernel	The cpufreq governor caps the maximum CPU frequency and may switch off some cores (hotplug); the GPU frequency is lowered; the screen's maximum brightness is reduced; the kernel idles sooner and uses a more aggressive autosleep; location polling by the GPS driver is reduced.
Runtime	The garbage collector and the JIT compiler run less often, and background compilation is postponed until charging; the runtime limits the number of background threads and defers non-urgent timers; animations may be disabled.
Application / framework	PowerManagerService restricts background services, jobs and syncs to fewer windows; only high-priority push remains; the app is told through PowerManager.isPowerSaveMode() and must reduce its own work: stop sensor sampling at a high rate, pause photo uploads, lower the refresh rate.

User-visible trade-off: Notes and fitness data refresh **less often** (for example sync every few hours instead of every few minutes) and animations are reduced.

Why this is acceptable: At 15% the primary goal is to keep the phone usable for calls and emergencies as long as possible. A delay in a non-urgent sync does not lose data, because it is only postponed and the app keeps the readings in local storage; the user can still force a manual refresh, and urgent push messages continue to be delivered.

Evidence provided: Four-state transition design, changed-assumption analysis (push-based wake-up) and a three-level battery-saver policy with a justified trade-off.

Task 6 – Design Challenge: RuralCare Field-Data Collection

Scenario: RuralCare collects patient records on three device classes: **Class A** modern Android smartphones, **Class B** iPhones, **Class C** legacy feature phones supporting only J2ME/JME applications or a WML browser. Workers are offline for hours, must finish a ten-hour shift on one charge, patient data must stay protected if a device is lost, and offline records must synchronise without loss or duplication.

Stage 1 – Interpret the Problem (2 Marks)

Question: Identify input, output, objective, constraints and decisions before choosing a technology.

Answer:

Element	Statement
Input	Patient records entered by field workers (identity, visit data, measurements, optional photos) on three device classes with very different runtimes (ART, native iOS, KVM-based J2ME/WML browser); intermittent connectivity.
Output	A central server database holding each record **exactly once**, complete and correct; local data protected on lost devices; a working device for the entire shift.
Objective	Collect, protect and synchronise patient records for all three device classes within the shift's battery budget, with no loss and no duplication.
Constraints	(1) Offline for hours: local storage and queueing needed. (2) Ten-hour shift on one charge: low power policy. (3) Lost devices: confidentiality of patient data at rest. (4) Sync without loss or duplication: unique IDs and idempotent

Element	Statement
	upload. (5) Class C has a restricted runtime, small storage, no secure storage and slow networks. (6) Patient data is sensitive: integrity and privacy.
Decisions to be made	The platform strategy for each class (native, cross-platform, web/WML); the local storage and encryption design; the sync protocol; the power policy; the failure handling.

Stage 2 – Propose the Strategy

Question: Consider the possible strategies, choose one and justify it from the characteristics of each class.

Answer:

Option	Principle	Suits the constraints?
O1 Native app per platform (A, B) plus J2ME for C	Each class uses its own best runtime and security features.	Yes for A and B: direct access to Keystore/Keychain, encrypted database, WorkManager/BackgroundTasks. Class C can only use J2ME. Costs three code bases.
O2 One cross-platform codebase for all	Shared code.	Works for A and B (for example one framework with native secure-storage plugins), but cannot run on Class C because J2ME has no such runtime.
O3 Web or WML-only for everyone	Server-based pages.	**Fails** the offline requirement on A and B, because a page needs a connection; WML on Class C also needs a connection for every entry.
O4 Smartphones only (A and B)	Exclude Class C.	Simplest, but the requirement states Class C devices exist in the field; excluding them removes coverage.

Chosen strategy: a hybrid – shared protocol, with a native client for Class A and B and a J2ME application with RMS storage for Class C, plus a WML/SMS fallback.

Justification from the device classes:

- **Class A and B** (modern runtimes with secure storage and background-task APIs) can store records encrypted in an on-device database for hours, hold the key in hardware-backed Keystore/Secure Enclave, and sync in batches. A cross-platform framework may be used for the user interface, but the security and sync layers use platform APIs. Offline operation is a hard constraint, so a web-only approach cannot satisfy it.
- **Class C**: J2ME (MIDP) apps can store data offline in the **Record Management System (RMS)** and use HTTP or SMS; a WML browser has no local storage beyond a cache, so WML alone fails offline operation. J2ME is the only way to capture data offline on those phones. WML/SMS is kept as an online **fallback** when a worker's phone cannot run the J2ME app.
- **A single common sync protocol** (record UUID, version, idempotent upload) is the shared element, so the server treats all three classes in the same way. This follows the principle that the strategy should match the problem's constraints (offline, protected, no duplication), not convenience or popularity.

Stage 3 – Identify the Critical Information

Question: Build a constraint table (one row per class) and identify the information the system must retain to avoid loss or duplication.

Answer – constraint table:

Class	Runtime environment	Available security mechanisms	Power management controls	Offline storage options	Connectivity
A – Android	ART (DEX, AOT/JIT, GC); NDK/JNI.	App sandbox (UID, SELinux); Android Keystore with TEE/StrongBox; file-based encryption; screen	Doze and App Standby; WorkManager/JobScheduler; battery saver; wake	SQLite (SQLCipher or Room with encryption), encrypted files;	3G/4G/Wi-Fi, intermittent; SMS

Class	Runtime environment	Available security mechanisms	Power management controls	Offline storage options	Connectivity
		lock and biometrics; TLS; remote wipe through device management.	locks with timeout; sensor batching.	gigabytes of storage.	fallback.
B – iOS	Objective-C/Swift native runtime, ARC, no VM.	App sandbox and entitlements; Keychain and Secure Enclave; Data Protection classes (file encryption tied to the passcode); mandatory code signing; ATS (TLS).	Suspended app model; BackgroundTasks and background fetch (system-scheduled); Low Power Mode.	SQLite/Core Data with Data Protection (NSFileProtectionComplete); gigabytes.	4G/Wi-Fi, intermittent.
C – Feature phone	J2ME/JME (CLDC + MIDP) on the KVM interpreter, or a WML browser (WAP).	Weak: MIDP sandbox and signed MIDlet domains; **no hardware key store**; limited APIs for cryptography (a lightweight library such as Bouncy Castle can be bundled); SIM-based PIN; HTTPS/WTLS depends on the handset.	Few controls: back-light timeout and radio sleep by the handset; the app can reduce screen use and network use; no background scheduling.	**RMS** (record stores, typically tens of KB to a few hundred KB); no file system on many phones; WML has none.	2G/GPRS, slow, or SMS only.

Information the system must retain to avoid loss or duplication:

Retained item	Why it is required
Globally unique record ID (UUID)	Lets the server recognise a record it has already

Retained item	Why it is required
generated on the device	received, so a repeated upload does not create a duplicate (idempotency).
Device ID and worker ID	Identifies the origin of each record for audit and for the unique key (device ID + local sequence number), and allows remote wipe of a lost device.
Local sequence number / timestamp and record version	Orders the records and resolves updates made to the same patient on two devices.
Sync status per record (pending, sent, acknowledged)	Tells the device which records have not yet been confirmed by the server, so none is lost after an interruption.
Last server-acknowledged sequence number (high-water mark)	Lets sync resume from the right point after an interruption without re-sending acknowledged records.
Integrity checksum (hash) of each record or batch	Detects corruption during transfer or storage.
The record itself, encrypted, kept until the server acknowledgement is received	Records must not be deleted locally until the server confirms them.

Stage 4 – Complete Solution Specification (4 Marks)

Question: Write a complete specification showing architecture per device class, data capture and offline storage, security design, power policy, synchronisation procedure and failure handling.

Answer:

4.1 Architecture for each device class

Class	Layers used	Runtime	Framework

Class	Layers used	Runtime	Framework
A	Kernel (Linux) → HAL → ART and native libraries (SQLite, BoringSSL) → Android framework → RuralCare app. Components: capture UI (Activity), storage layer (encrypted SQLite), crypto layer (Keystore), sync layer (WorkManager job, HTTPS client).	ART	Android SDK: Activity, WorkManager, Keystore, ContentProvider.
B	XNU kernel → Core OS (Security, Keychain) → Core Services (Foundation, Core Data, URLSession) → Cocoa Touch (UIKit, BackgroundTasks) → RuralCare app. Same four app components.	Objective-C/Swift native runtime with ARC	Cocoa Touch.
C	Phone OS (vendor RTOS) → KVM/CLDC → MIDP → RuralCare MIDlet. Components: capture form (LCDUI), RMS storage, lightweight crypto library, sync over HTTP or SMS. Fallback: WAP/WML browser pages served by the server for online entry.	KVM interpreter (J2ME)	CLDC + MIDP (LCDUI, RMS, GCF HttpConnection).

4.2 Data capture and offline storage design

- Each visit creates a **record** with fields {recordUUID, deviceID, workerID, seqNo, version, timestamp, payload, checksum, status}.
- **Class A/B:** record saved immediately (on every field-group completion) to the encrypted local database; status = PENDING. Photos are compressed and stored as encrypted files referenced by the record.
- **Class C:** record stored in an RMS record store PENDING; payload limited to key fields (text and numbers only, no photos); the oldest ACKNOWLEDGED records are deleted when the store reaches 80% capacity.
- Auto-save every step so a battery shutdown loses at most the field currently being typed.

4.3 Security design

Aspect	Class A	Class B	Class C
Data at rest	AES-256 encrypted database; key generated in Android Keystore (hardware-backed, requires user authentication to use); file-based encryption of the device.	AES-256 encrypted database with the key in the Keychain (accessible only when unlocked); Data Protection class complete.	Records encrypted with AES using a key derived (PBKDF2) from the worker's **PIN**, with a bundled lightweight crypto library; the key is not stored on the phone; PIN lock after 3 failed attempts wipes the RMS store.
In transit	HTTPS (TLS 1.2+), certificate pinning; each batch signed with a device key (HMAC).	Same, through ATS.	HTTPS where the handset supports it; otherwise the payload is **encrypted at application level** (AES with a shared server key) before sending over HTTP or SMS.
If the device is lost	App lock (biometrics or PIN); **remote wipe** by deleting the Keystore key (crypto-erase), so the encrypted data becomes unreadable; the server revokes the device token.	Same: Keychain key invalidated; Find My / MDM wipe; the server revokes the token.	Data is unreadable without the PIN (encrypted at rest); server revokes the device ID; no remote wipe exists, so the PIN protection and limited record count are the safeguards.

4.4 Power management policy for a ten-hour shift

Rule	Detail
Radio duty cycle	Radios stay idle while collecting; one sync attempt on connectivity return, and then at most one scheduled sync per 2 hours (batched).
No wake locks or polling	The app holds no wake lock outside active form entry; no background polling (A/B); the OS schedules sync (WorkManager with network constraint; BackgroundTasks).
Screen	Dim and time out the screen after 30 seconds; dark theme where an OLED display exists; no animations.
GPS and sensors	GPS only when a record needs a location, one fix only; no continuous tracking.
Thresholds	At 30% charge: shorten the screen timeout and stop photo upload. At 15%: battery saver; sync only text; at 5%: flush and save all data, then show a warning (and a final sync attempt if the network is available).
Class C	Screen back-light timeout set to a minimum; the app sends one batched message at sync time; no network use otherwise.

4.5 Synchronisation procedure (ordered steps)

1. **Detect** connectivity (A/B: the OS callback; C: the worker presses *Sync* or the MIDlet tries on start-up).
2. **Authenticate** with the server (TLS plus device token; Class C: signed token or PIN-derived code).
3. **Select** all records with status PENDING in sequence order; build a batch (A/B: up to 50 records; C: up to 5 records per request).

4. **Compute** a batch checksum; send the batch with deviceID and the first and last seqNo.
5. The server **verifies** the checksum, then inserts each record using its UUID as the unique key (**insert-if-absent**; an existing UUID returns "already stored" without any change).
6. The server returns an **acknowledgement** listing the UUIDs stored (or already stored) and the new high-water mark.
7. The device marks those records ACKNOWLEDGED, updates the high-water mark and repeats from step 3 until none remain.
8. Acknowledged records are deleted from the device after a retention delay (for example 24 hours), never before.

Interrupted sync: If the connection fails before step 6, the device receives no acknowledgement, so the records remain PENDING. On the next attempt they are sent again; because the server uses the UUID as the unique key, a second copy is recognised, so there is **no duplication and no loss**. A partially received batch is rolled back by the server's transaction (all-or-nothing per batch).

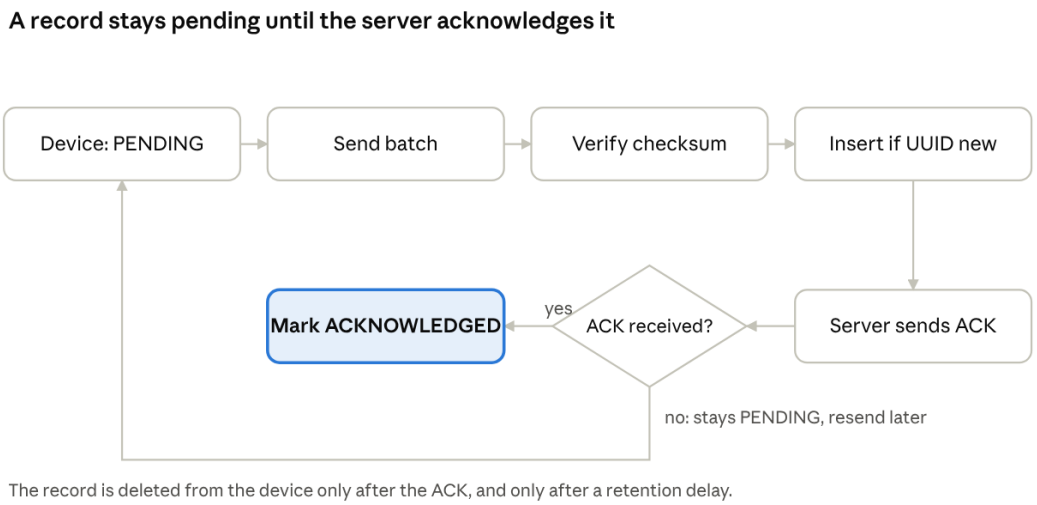


Figure 3 · sync loop with acknowledgement · 6 steps, 1 decision

Figure 3. The synchronisation procedure of Stage 4.5 drawn as a loop. The "no" branch is why an interrupted sync cannot lose a record, and the UUID check on the server is why the repeat cannot duplicate it.

4.6 Failure handling for each device class

Failure	Class A	Class B	Class C
Battery dies mid-entry	Auto-saved partial draft in an encrypted DB; recovers at restart.	Same (Core Data saves on every step).	RMS write per field group; the draft is recovered at start.
Storage full	Warn at 90%; stop photo capture; text records continue.	Same.	Delete the oldest ACKNOWLEDGED records; if none exist, force a sync prompt and block new records only when nothing can be freed.
App killed by the OS	Records already in the DB; WorkManager restarts sync.	Same; the background task resumes when scheduled.	Not applicable (no OS killing); the MIDlet restarts from RMS.
Network never returns	Records stay encrypted locally; the supervisor can collect them by a manual export over USB or Wi-Fi Direct (encrypted).	Same.	Fallback: send each record as an encrypted SMS to a gateway; the worker presses *Send by SMS*.
Corrupted data	Checksum failure triggers a re-send; the corrupted local row is quarantined.	Same.	Checksum failure: the record is re-entered or re-sent.
Device lost	See 4.3.	See 4.3.	See 4.3.

Stage 5 – Desk-Check (1 Mark)

Question: A Class A worker is offline for six hours, the battery falls to 20%, and the phone is then lost before any synchronisation. Show the states, decisions and protective actions; report what is exposed, lost and recoverable.

Answer:

Step	Time	State and decision	Protective action
1	0 to 6 h	Worker records visits; each record saved encrypted with status PENDING; the sync job is waiting for connectivity.	Keystore key protects the database; the screen locks after 30 s.
2	At 30%	Reduced-power rules begin: shorter screen timeout, no photo upload.	–
3	At 20%	Not yet at 15%; the app continues normal collection, with no sync possible (no network).	–
4	After 6 h	The phone is lost. All records are PENDING and exist only on the device.	The device is locked; Keystore key requires the user's screen lock.
5	Supervisor reports the loss	Server revokes the device token; an MDM remote wipe command is queued, and the Keystore key is deleted when the phone next connects.	Crypto-erase makes the data unreadable.

Result category	Outcome
Data exposed	Essentially none: the database is AES-256 encrypted with a hardware-backed key that is released only after screen-lock authentication. The possible exposure is to someone who can unlock the phone (known PIN) before the wipe. The metadata

Result category	Outcome
	visible on the lock screen should be none, because notifications show no patient data.
Data lost	The records collected in the six hours are not on the server, so if the phone is never recovered they are **lost to RuralCare** (their only copy was on the device). This is the residual risk of operating offline.
Recoverable	Everything synchronised before the shift; the server's list of already-acknowledged records; if the phone is recovered, the encrypted records can still be synchronised (UUID prevents duplication). Mitigation: the worker syncs at the next connectivity opportunity during the shift, so that the amount at risk is small.

Stage 6 – Attack Your Own Design (2 Marks)

Question: Construct at least three adverse conditions, analyse why each is challenging, predict the behaviour, apply them, and determine whether the design survives, which decision is critical and whether any assumption fails.

Answer:

#	Adverse condition	Why it is challenging	Prediction	Result of applying it
C1	**Stolen unlocked phone** (worker was using it when it was snatched).	The Keystore key is available while the device is unlocked, so encryption at rest does not protect the data; the thief has the	The design will partly fail: the thief can read recent records until the screen locks.	Result: exposure is limited by the **30-second auto-lock** and by requiring a **re-authentication (biometric or PIN) in the app** to open patient lists; the key is configured to need user authentication within a short window. Exposure is reduced to the in-memory session. **Refinement: add an in-app**

#	Adverse condition	Why it is challenging	Prediction	Result of applying it
		same rights as the worker.		**PIN timeout of 60 seconds and a remote revoke.**
C2	**Corrupted sync mid-transfer** (the connection drops while the server is writing a batch).	A half-written batch could create partial records or a lost acknowledgeme nt.	The design will survive: the batch is atomic, and the device resends.	Result: the server's transaction rolls back; the device receives no acknowledgement and keeps PENDING; on retry, UUID idempotency prevents duplicates. **The critical decision is the UUID-based, insert-if-absent upload with delayed local deletion.**
C3	**Class C phone with no secure storage** and a thief with a PC and an RMS reader.	There is no hardware key store, so any key stored on the phone could be extracted.	The design should survive only if the key is not stored on the phone.	Result: the key is derived from the **worker's PIN** and never stored; a four-digit PIN can be brute-forced offline. **Assumption that fails:** a 4-digit PIN gives enough protection. **Refinement:** require a 6+ character PIN, apply PBKDF2 with a high iteration count, add the wipe after 3 failed attempts (effective only through the app, not offline), and limit records on the phone.
C4	**Two devices edit the same patient**	Each device creates its own records, and later versions may conflict.	The design will keep both versions rather than	Result: records have unique UUIDs and a version/timestamp; the server stores both and flags the conflict for review by a supervisor instead of overwriting. No loss, no duplicate.

#	Adverse condition	Why it is challenging	Prediction	Result of applying it
	while both are offline.		lose one.	
C5	**Battery dies at 0% during a write.**	Interrupted write may corrupt a record.	Survives: draft-level saves and checksums.	Result: partial records are discarded or recovered from the last saved draft; completed records are intact.

Conclusion: The design meets the requirements in four of the five cases without change. The **critical decisions** are (1) encryption at rest with a key released only after user authentication, and (2) idempotent, acknowledgement-based synchronisation with a UUID. The assumptions that fail are that an **unlocked phone is safe** (C1) and that a **short PIN protects Class C** (C3); both are repaired by the refinements stated.

Summary and References

Task	Marks	Key result
1 Architecture review	6	Eight deficiencies found; Android specified as six layers with kernel, runtime, security and power located.
2 Cross-platform comparison	8	Process model and background-execution policy matter most; push (FCM, APNs, WNS) is the delivery path.
3 Runtime analysis	9	Ten-step launch sequence; AOT native with reference counting suits low-memory devices.
4 Security forensics	7	Four mechanisms restored: permissions, sandbox/Keystore, code signing, TLS.

Task	Marks	Key result
5 Power management	7	Four-state policy, push-based wake-up, battery saver at 15%.
6 RuralCare design	13	Hybrid strategy; UUID idempotent sync; encrypted offline storage; five adverse conditions tested.

Note on Figure 2: the battery curves are an illustrative model with assumed currents, not measured data. Replace the assumptions if your course provides figures.

References

- Android Open Source Project documentation: architecture, Binder, ART, SELinux, Doze and App Standby.
- Apple Developer documentation: App Sandbox, Keychain, Data Protection, BackgroundTasks, Darwin/XNU overview.
- Microsoft documentation: Windows Runtime (WinRT), AppContainer, background tasks.
- Course material for ITA03 Mobile Computing, CO4 (mobile OS architecture, runtimes, security, power management; J2ME/WML).